

Recursion and Backtracking – Complete Documentation

Prepared for DSA Learning & Interview Preparation

1. Recursion

Recursion is a programming technique where a function calls itself to solve a problem by breaking it into smaller subproblems. Every recursive function must have a base case to stop execution and a recursive case to continue the breakdown.

Key Characteristics:

- Function calls itself
- Solves smaller instances of the same problem
- Uses call stack implicitly
- Must contain a base condition

Examples: Factorial, Fibonacci, Tree Traversal, Binary Search

2. Backtracking

Backtracking is an algorithmic technique that incrementally builds solutions and abandons a path as soon as it determines that the path cannot lead to a valid solution.

Key Characteristics:

- Uses recursion internally
- Explores all possibilities
- Has an undo (backtrack) step
- Efficient through pruning

Common Applications: Sudoku, N-Queens, Permutations, Maze Problems

3. Relationship Between Recursion and Backtracking

Backtracking is a specialized use of recursion. While recursion focuses on solving subproblems, backtracking focuses on exploring all valid possibilities using recursion. Not all recursion problems involve backtracking, but all backtracking solutions are implemented using recursion.

4. Types of Backtracking

4.1 Decision / Choice-Based Backtracking

In this type, at each step, a binary decision is made such as include or exclude an element.

Examples: Subsets, Subsequences, Binary Strings

4.2 Permutation-Based Backtracking

This type generates all possible arrangements where the order of elements matters.

Examples: String permutations, Array permutations

4.3 Combination-Based Backtracking

In combinations, order does not matter. A start index is often used to avoid duplicates.

Examples: Combination Sum, Choosing k elements

4.4 Constraint-Based Backtracking

Each step checks whether the current choice satisfies constraints. Invalid paths are pruned early.

Examples: N-Queens, Sudoku, Knight's Tour

4.5 Path-Finding Backtracking

This type explores all possible paths from a source to a destination in grids or graphs.

Examples: Rat in a Maze, Grid Paths, Word Search

4.6 Optimization Backtracking

Used to find the best possible solution such as minimum cost or maximum profit.

Examples: Minimum cost path, Scheduling problems

5. Interview Tips & Problem Identification

- If order matters → Permutation backtracking
- If yes/no decision → Decision-based backtracking
- If constraints exist → Constraint-based backtracking
- If grid or path → Path-finding backtracking

Mastering these patterns allows quick identification and efficient solving of backtracking problems in interviews.